Assignment 3: Multiplication / Division Units

> **Info**
>
> Assignment 2 consists of 1 task, with 2 subtasks, for a total of **30 points**.
>
> Assignment 2 is a group exercise. You will be assessed as a group, but scored individually.
>
> There could still be minor updates, which will be highlighted.

Task 1: Implementing Division (10 points)

For Task 1, we will incorporate **both signed and unsigned division** into the given `MCycle` unit [HDL simulation only].

- The design files can be found here. Please go through the comments carefully to understand the operation of the unit.
- Implement both **signed and unsigned division** in the `MCycle` unit.
- Simulate the unit using a good testbench covering appropriate corner cases.
- Synthesize the `MCycle` unit by setting it as the top-level module and make sure it synthesizes without warnings (unless you are sure a warning can be safely ignored) before proceeding to the next task of incorporating it into the processor.
- We can assume that the divisor is never zero.

Task 2: Incorporating MCycle into Our CPU (5 points)

For Task 2, we must incorporate the `MCycle` unit into our processor so that it can execute the 32-bit RISC-V `mul`, `div`, and `divu` instructions. **HDL simulation and hardware implementation are both required.**

- `mul`, `div`, and `divu` are part of the RISC-V M extension. Implement their 32-bit versions.
- `div` performs signed division, while `divu` performs unsigned division.
- The destination register should contain the quotient for `div` and `divu`. The remainder can be discarded.
- Since `mul` writes only the lower 32 bits of the result, there is no difference between signed and unsigned multiplication for this instruction.

- The control unit will need to be modified to generate the `Start` and `MCycleOp` control signals.
- `!Busy` can be used as the write enable for the PC. This will stall the processor until the multicycle operation is complete.
- The datapath should be modified to make the appropriate connections to and from the `MCycle` unit. We will need a multiplexer and a control signal to combine the outputs from the ALU and `MCycle`.
- `mulh` variants (upper word) and `rem` variants (remainder) are not required, though you may implement them if you wish.

Task 3: Enhancement (5 points)

Improve **one** of the following units in the `MCycle` implementation:

- signed multiplication; *or*
- signed division; *or*
- unsigned division.

The objective is to explore a meaningful improvement in **execution time, hardware cost, or the trade-off between the two** [post-synthesis simulation; showing the enhancement on hardware is left to your discretion].

Some suggestions for improvement are given below. You need not do all of them. **We will evaluate only one improvement.** The purpose of this task is to incentivize some exploration, and **spending too much time on it is not recommended**, as the task carries only 5 points.

- Explore techniques that trade additional hardware for fewer execution cycles. For example, an implementation might process more than one bit per iteration, reducing the number of cycles required for multiplication or division.
- Reduce hardware cost by sharing a single adder between multiplication and division within the `MCycle` unit. You could take this one step further by reusing the adder from the ALU, thus saving another adder, although this will require substantially more effort and is unlikely to be worthwhile for 5 points. The latter approach will also require modifications to the ports of the `MCycle` unit.
- Implement Booth's multiplication algorithm or another efficient multiplication/division algorithm that you find in the literature or online.
- Other meaningful architectural or algorithmic improvements are welcome. You should be able to explain clearly what has been improved and demonstrate the effect of the enhancement.
- **DO NOT implement a single-cycle multiplier using the `*` operator.** FPGAs contain

built-in multipliers/DSP units that are inferred when we use the `*` operator. These are much more efficient than array-adder-based multipliers that we could implement ourselves, but using them defeats the learning objectives of this assignment.

Task 4: Visualisation of Enhancement (5 points)

Use AI/LLMs to build an interactive visualisation tool for **the specific multiplier or division enhancement that you implemented in Task 3**. The tool should be implemented as a **single HTML file containing HTML, CSS, and JavaScript** and should serve as a learning aid for understanding the enhanced unit.

The visualisation should be designed for someone who is not already familiar with the algorithm or implementation. It should help the user understand both the underlying computation and how the hardware performs it. The emphasis should therefore be on an **interactive and visual explanation**, rather than simply presenting the algorithm as text.

Users should be able to enter operands and step through the computation **cycle by cycle or iteration by iteration, as appropriate to your implementation**. The visualisation should clearly show:

- the state of the relevant registers and other important datapath elements;
- the operation being performed at each step;
- how the state changes from one step to the next;
- the cycle count, etc.

The visualisation should correspond to the **actual enhancement implemented for Task 3**, rather than being a generic visualisation of multiplication or division.

Provide the complete history of prompts used to create the visualisation, together with any skills files or other relevant Markdown files used.

You are expected to understand the algorithm, but not the HTML/CSS/JavaScript code.

Design Instructions

- You are required to have your own, comprehensive program to have a convincing demo. **Only one assembly language program (and hence one bitstream) will be allowed for the demo.**
- While not a formal requirement, we suggest converting test_MCycle into a self-checking testbench, possibly with AI assistance (but understand the generated code).

- If you are using the UART console, you can set the radix to hexadecimal in the "Display" tab of RealTerm which may make things easier.
- Remember to use plenty of test cases to verify that your multiplication and division implementations work as intended.

Submission Info

- Assignment 3 will be evaluated in **Week 9**. The presentation schedule can be found on Canvas.
- Please upload the Assignment 3 files to Canvas by the stipulated deadline (generally before your lab time), including the following files:
- `.v` / `.vhd` files you have created or modified [RTL sources and testbench(es)];
- `.bit` file;
- `.s` / `.asm` file containing your assembly program;
- `.ppt` or `.pdf` file — 1 to 4 slides explaining and demonstrating the enhancement implemented for Task 3;
- the single-file `.html` visualisation developed for Task 4; and
- a text/Markdown file containing the complete prompt history used for Task 4, together with any other relevant Markdown files (skills, etc.), if any.

Place these files in an archive with the filename `GroupXX_Monday/Friday_Asst3.zip` (replace `XX` with your group number) and upload it to Canvas.

One submission per group is sufficient. If there are multiple submissions, the file with the latest timestamp will be taken as the final submission.

Do not zip and upload the complete project folder. Only the files mentioned above should be included. **The submitted implementation files should be the exact same files used for the demo.**